

Redes Neuronales Recurrentes Análogas con Pesos Reales

Andrés Sicard Ramírez
asicard@eafit.edu.co

Juan C. Agudelo Agudelo
jagudelo@eafit.edu.co

Mario E. Vélez Ruiz
mvelez@eafit.edu.co

Grupo de Lógica y Computación
Escuela de Ciencias y Humanidades
Universidad EAFIT; Medellín, Colombia

Resumen

Se definen los circuitos booleanos. Se define como decidir lenguajes por medio de familias de circuitos booleanos y se demuestra que existen familias de circuitos booleanos que deciden lenguajes no recursivos y por lo tanto son modelos de hipercomputación. Luego, se definen las redes neuronales recurrentes análogas (ARNNs), se define como decidir lenguajes por medio de ARNNs y por último se demuestra que todo lenguaje decidido por una familia de circuitos booleanos puede ser decidido por una ARNN con pesos reales y por lo tanto algunas ARNNs son modelos de hipercomputación.

Abstract

We define boolean circuits. We define how decide languages by boolean circuits families, and then we demonstrate that boolean circuits families decide nonrecursive languages and therefore they are hipercomputation models. Then we define analog recurrent neural networks (ARNNs), how decide languages by ARNNs and at last we demonstrate that all languages decide by boolean circuits families can be decide by ARNNs with real weights and therefore some ARNNs are hipercomputation models.

1 Circuitos Booleanos

Los circuitos booleanos son un modelo de computación por medio del cual se pueden computar funciones booleanas, es decir, funciones de la forma $f: \{0,1\}^n \rightarrow \{0,1\}^k$, o se pueden decidir lenguajes (determinar si una palabra pertenece o no a un lenguaje) sobre el alfabeto $\Sigma = \{0,1\}$. En el presente artículo sólo se presenta la definición para la decisión de lenguajes, para un tratamiento sobre el cómputo de funciones mediante circuitos booleanos ver [8, 13, 15].

Definición 1 (Circuito Booleano). *Un circuito booleano es un grafo dirigido acíclico $C = (V, E)$ donde los nodos en $V = \{1, \dots, n\}$ son denominados ‘Compuertas’ (‘gates’) de C . Al número de aristas que llegan a un nodo se le denomina ‘fan-in’ y al número de aristas que salen se le denomina ‘fan-out’. Cada compuerta $i \in V$ tiene un tipo asociado $s(i)$, donde $s(i) \in \{\mathbf{true}, \mathbf{false}, \wedge, \vee, \neg\} \cup \{x_1, x_2, \dots\}$ ¹. Si $s(i) \in \{\mathbf{true}, \mathbf{false}\} \cup \{x_1, x_2, \dots\}$ entonces el fan-in de i debe ser 0. Las compuertas con fan-in 0 son denominadas ‘entradas del circuito’. Las compuertas con fan-out 0 son denominadas ‘salidas del circuito’. Si $s(i) = \neg$, entonces i debe tener fan-in 1. Si $s(i) \in \{\wedge, \vee\}$ entonces i debe tener fan-in mayor o igual² a 2. Como*

¹Los valores **true** y **false** son representados por 1 y 0 respectivamente.

²En [8] se restringe el fan-in de las compuertas $\{\wedge, \vee\}$ a 2, pero en [10] se trabaja sin esta restricción.

el grafo C es acíclico puede ser visto por niveles. Las compuertas del nivel j sólo reciben entradas de las compuertas de niveles inferiores i , $i < j$. La ‘profundidad’ del circuito booleano es el número de niveles menos 1, y el ‘tamaño’ del circuito booleano es el número total de compuertas [8, 10, 13].

Ejemplo 1. La figura (1) representa un ejemplo de circuito booleano.

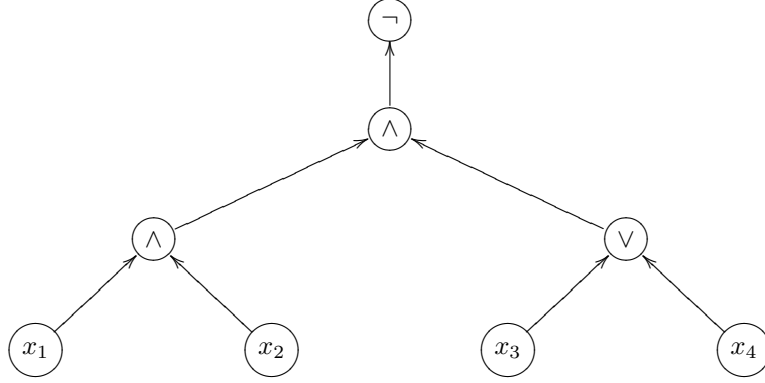


Figura 1: Circuito Booleano.

Definición 2 (Familia de Circuitos Booleanos). Una familia de circuitos $\mathcal{F}$ es un conjunto indexado

$$\{C_n \mid n \in \mathbb{N}\},$$

donde C_n es el circuito que computa las entradas de longitud n [10].

Definición 3 (Lenguaje Decidido por una Familia de Circuitos Booleanos). Sea $\mathcal{L} \subseteq \{0, 1\}^*$ un lenguaje sobre el alfabeto $\Sigma = \{0, 1\}$ y $\alpha = a_1 a_2 \dots a_n$ una palabra de Σ^* . Se dice que la familia de circuitos booleanos $\mathcal{F}$ (donde cada C_n tiene sólo una compuerta de salida) decide el lenguaje $\mathcal{L}$ si y sólo si el valor de la compuerta de salida del circuito $C_n \in \mathcal{F}$ con las entradas $a_1 a_2 \dots a_n$ es 1 si $\alpha \in \mathcal{L}$ y 0 si $\alpha \notin \mathcal{L}$. [10].

Ejemplo 2. La familia de circuitos booleanos $\mathcal{F} = \{C_n \mid n \in \mathbb{N} \wedge C_n = \bigwedge_{i=1}^n x_i\}$ decide el lenguaje $\mathcal{L} = \{1\}^*$. Los circuitos de esta familia son representados por la figura (2).

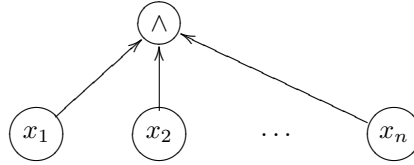


Figura 2: Circuitos de la familia $\mathcal{F} = \{C_n \mid n \in \mathbb{N} \wedge C_n = \bigwedge_{i=1}^n x_i\}$.

Ejemplo 3. La familia de circuitos booleanos $\mathcal{F} = \{C_n \mid n \in \mathbb{N} \wedge C_n = \bigvee_{i=1}^n x_i\}$ decide el lenguaje $\mathcal{L} = \{0, 1\}^* 1 \{0, 1\}^*$, los circuitos de esta familia son representados por la figura (3).

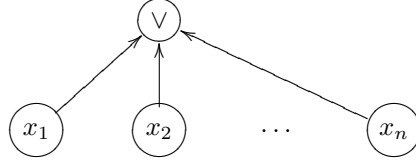


Figura 3: Circuitos de la familia $\mathcal{F} = \{C_n \mid n \in \mathbb{N} \wedge C_n = \bigvee_{i=1}^n x_i\}$.

Definición 4 (Familia de Circuitos Booleanos Uniformes). *Se dice que la familia de circuitos booleanos $\mathcal{F} = \{C_n \mid n \in \mathbb{N}\}$ es uniforme si existe una máquina de Turing $\mathcal{M}$ que recibe como entrada 1^n y produce como salida C_n [8, 13].*

Ejemplo 4. *Las familias de circuitos booleanos presentadas en los ejemplos (2) y (3) son familias de circuitos booleanos uniformes, es posible construir una máquina de Turing que reciba como entrada 1^n y produzca como salida el circuito booleano C_n representado por las figuras (2) y (3) en alguna codificación específica.*

Definición 5 (Familia de Circuitos Booleanos no Uniformes). *Se dice que la familia de circuitos booleanos $\mathcal{F} = \{C_n \mid n \in \mathbb{N}\}$ es no uniforme si no es posible construir una máquina de Turing $\mathcal{M}$ que reciba como entrada 1^n y produzca como salida C_n [8, 13].*

Ejemplo 5. *Sea $\mathcal{N}$ un conjunto no recursivo de números naturales. Se define la familia de circuitos booleanos $\mathcal{F} = \{C_n \mid n \in \mathbb{N}\}$ donde:*

$$C_n = \begin{cases} \bigwedge_{i=1}^n (x_i \vee \neg x_i) & \text{si } n \in \mathcal{N}, \text{ (figura 4);} \\ \bigwedge_{i=1}^n (x_i \wedge \neg x_i) & \text{si } n \notin \mathcal{N}, \text{ (figura 5).} \end{cases} \quad (1)$$

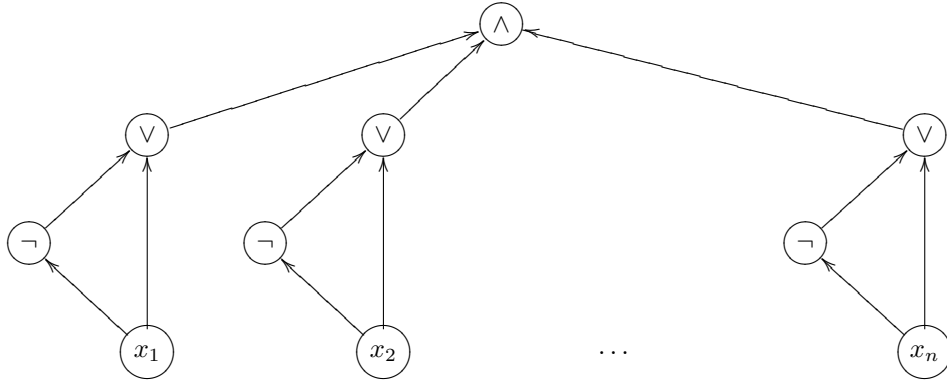


Figura 4: Circuitos $C_n = \bigwedge_{i=1}^n (x_i \vee \neg x_i)$.

No se puede construir una máquina de Turing que reciba como entrada 1^n y produzca como salida C_n , debido a que ninguna máquina de Turing puede determinar cuando n pertenece o no a $\mathcal{N}$ ($\mathcal{N}$ es un conjunto no recursivo) y por consiguiente no puede determinar si C_n es $\bigwedge_{i=1}^n (x_i \vee \neg x_i)$ ó $\bigwedge_{i=1}^n (x_i \wedge \neg x_i)$, por lo tanto $\mathcal{F}$ es una familia de circuitos booleanos no uniforme.

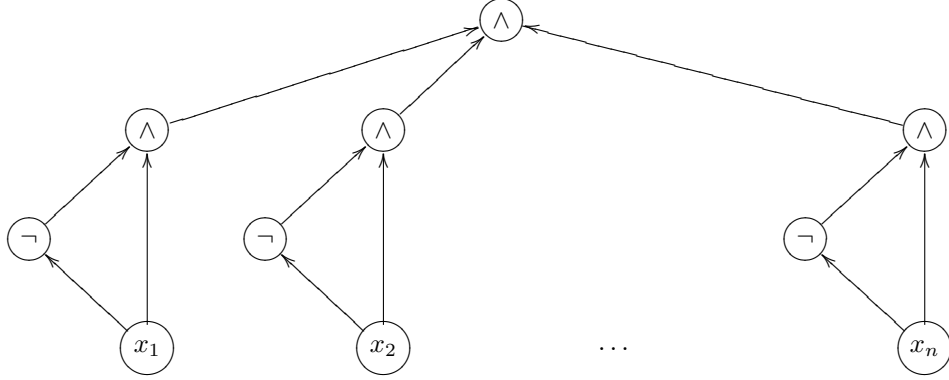


Figura 5: Circuitos $C_n = \bigwedge_{i=1}^n (x_i \wedge \neg x_i)$.

Definición 6 (Máquina de Turing). *Informalmente una máquina de Turing está compuesta por una cabeza de lectura-escritura, una cinta unidimensional bi-infinita dividida en celdas y un conjunto finito de instrucciones, en todo instante discreto de tiempo la máquina se encuentra en un estado determinado (de un conjunto finito de estados) y leyendo un símbolo sobre la cinta, de acuerdo a esta situación la máquina selecciona una instrucción (de un conjunto finito de instrucciones) y la ejecuta escribiendo un símbolo sobre la cinta, realizando un movimiento (hacia la derecha, izquierda, o no movimiento) y pasando a otro estado (posiblemente el mismo).*

Formalmente una máquina de Turing está definida por la estructura matemática:

$$\mathcal{MT} = \langle Q, \Sigma, M, I \rangle, \text{ donde :}$$

$Q = \{q_0, q_1, \dots, q_n\}$: Conjunto finito de estados de la máquina ($Q \neq \emptyset$).

$\Sigma = \{s_0, s_1, \dots, s_m\}$: Alfabeto o conjunto finito de símbolos de entrada-salida, adoptando por convención $s_0 = \square$, (símbolo vacío) ($\Sigma - \{s_0\} \neq \emptyset$).

$M = \{L, R, N\}$: Conjunto de movimientos (L : Izquierda, R : Derecha, N : No movimiento).

$I = \{i_0, i_1, \dots, i_p\}$: Conjunto finito de instrucciones de la forma $i_j = q_m s_m s_n m q_n$, donde $q_m, q_n \in Q$; $s_m, s_n \in \Sigma$ y $m \in M$. Cuando la máquina esta en el estado q_m leyendo el símbolo s_m ejecuta la instrucción i_j , escribiendo el símbolo s_n realizando el movimiento m y pasando al estado q_n . Las instrucciones también pueden ser vistas como una relación de $Q \times \Sigma$ en $\Sigma \times M \times Q$, si existe más de una instrucción para alguna pareja $q_m, s_m \in \{Q \times \Sigma\}$ la máquina de Turing es ‘no determinista’, si en las instrucciones no se repite la pareja $q_m s_m$ la máquina de Turing es ‘determinista’ [14, 4, 6, 5].

Definición 7 (Lenguaje Decidido por una Máquina de Turing). *Sea Σ una alfabeto de una máquina de Turing $\mathcal{MT}$ y sea $\mathcal{L} = (\Sigma - \{\square\})^*$ un lenguaje. Se adiciona $\{q_Y, q_N\}$ a los estados de $\mathcal{MT}$ y se dice que $\mathcal{MT}$ decide el lenguaje $\mathcal{L}$ si para todo $\alpha \in (\Sigma - \{\square\})^*$ se cumple:*

1. Si $\alpha \in \mathcal{L}$ entonces $\mathcal{MT}$ se detiene en el estado q_Y .

2. Si $\alpha \notin \mathcal{L}$ entonces $\mathcal{MT}$ se detiene en el estado q_N .

Si existe $\mathcal{MT}$ que decida el lenguaje $\mathcal{L}$, se dice que el lenguaje $\mathcal{L}$ es ‘recursivo’.

Definición 8 (Modelo de Hipercomputación). Se les denomina modelos de hipercomputación a los modelos de computación que pueden computar objetos que no pueden ser computados en una máquina de Turing [2, 3]. En este contexto, la computación de objetos se refiere a la decisión de lenguajes, por lo tanto, un modelo de hipercomputación será un modelo que decida lenguajes que no pueden ser decididos por una máquina de Turing.

Teorema 1. Existen familias de circuitos booleanos no uniformes que son modelos de hipercomputación.

Demostración. La familia de circuitos booleanos $\mathcal{F}$ del ejemplo (5) decide el lenguaje $\mathcal{L} = \{\alpha \in \{0, 1\}^* \mid |\alpha| \in \mathcal{N}\}$, donde $|\alpha|$ significa la longitud de α . $\mathcal{L}$ es un lenguaje no recursivo, debido a que $\mathcal{N}$ es un conjunto no recursivo, por lo tanto $\mathcal{F}$ es un modelo de hipercomputación. $\square$

2 Redes Neuronales Recurrentes Análogas

Definición 9 (Red Neuronal Recurrente Análoga). Una ARNN (Analog Recurrent Neural Network) [10] es una red neuronal compuesta por N procesadores elementales llamados neuronas, cada neurona tiene asociado un valor de activación $x_i(t)$, en cada instante discreto de tiempo t la red neuronal recibe M entradas binarias $u_j(t)$. La dinámica de la red consiste en calcular los valores de activación de las neuronas de acuerdo a las entradas y los valores de activación de las neuronas en el instante de tiempo anterior. Cada neurona calcula su valor de activación de acuerdo a la ecuación:

$$x_i(t+1) = \sigma \left(\left(\sum_{j=1}^N a_{ij} \cdot x_j(t) \right) + \left(\sum_{j=1}^M b_{ij} \cdot u_j(t) \right) + c_i \right), \quad (2)$$

donde a_{ij} es el peso del enlace entre la neurona x_j y la neurona x_i , b_{ij} es el peso del enlace entre la neurona u_j y la neurona x_i y c_i es el peso constante asociado a la neurona x_i . La función σ es llamada “función de activación” o “función de respuesta” y el argumento que recibe es una función de las entradas a la neurona llamada “función de red”. En cada instante de tiempo la salida de la red es el valor de activación de un subconjunto de ℓ neuronas $y_1(t), \dots, y_\ell(t)$.

Los valores de activación de las neuronas son representados por un vector $\mathbf{X}(t)$ de dimensión $N \times 1$, las entradas por un vector $\mathbf{U}(t)$ de dimensión $M \times 1$, los pesos de los enlaces entre las neuronas por una matriz $\mathbf{A}$ de dimensión $N \times N$, los pesos de los enlaces entre las entradas y las neuronas por una matriz $\mathbf{B}$ de dimensión $N \times M$ y los pesos constantes por un vector $\mathbf{C}$ de dimensión $N \times 1$. De esta forma se representa la dinámica total de la red por la ecuación matricial:

$$\mathbf{X}(t+1) = \sigma(\mathbf{A} \cdot \mathbf{X}(t) + \mathbf{B} \cdot \mathbf{U}(t) + \mathbf{C}). \quad (3)$$

Definición 10 (Computación en una ARNN). Una computación en una ARNN es una sucesión de cálculos en instantes discretos de tiempo de los valores de activación $\mathbf{X}(t+1)$ de las neuronas de acuerdo a los valores de activación $\mathbf{X}(t)$ y las entradas $\mathbf{U}(t)$ en el instante de tiempo anterior, teniendo en cuenta los enlaces y pesos de la red $\mathbf{A}, \mathbf{B}$ y $\mathbf{C}$, de acuerdo a la ecuación (3). Las salidas en cada iteración son un subconjunto de los valores de activación calculados $y_1(t), \dots, y_\ell(t)$. Al inicio de la computación, tiempo

$t = 0$, cada neurona tiene un valor de activación inicial $x_i(0)$ (normalmente $x_i(0) = 0$ para $i = 1, 2, \dots, N$). La computación termina cuando no hay más entradas.

Hay que tener en cuenta que en la mayoría de ARNNs se introduce un retardo r en la computación, es decir, los primeros r valores de activación de las neuronas de salida no deben ser tomados en cuenta como parte del cómputo, esto se debe a que las entradas por lo general no llegan directamente a las neuronas de salida y para llegar a afectarlas tienen primero que propagarse por otras neuronas de la red, este tiempo de retardo depende de la estructura específica de la red.

Definición 11 (Lenguajes Decididos por una ARNN). La definición de computación presentada en [10] se enfoca en redes con un protocolo de entrada-salida en la que se tienen sólo dos líneas de entrada $D(t)$ y $V(t)$; $D(t)$ es la entrada por donde se ingresan los datos a computar y $V(t)$ sirve de línea de validación de la entrada indicando cuando $D(t)$ está activa o no. De forma similar, sólo se tienen dos líneas de salida, una para la salida de datos y otra para la validación de la salida, denotadas por $G(t)$ y $H(t)$ respectivamente. Teniendo en cuenta este protocolo, para decidir lenguajes sobre el alfabeto $\Sigma = \{0, 1\}$ se tiene que las entradas a la ARNN para la palabra $\alpha = a_1 a_2 \dots a_n \in \{0, 1\}^*$ a ser decidida son:

$$D(t) = \begin{cases} a_t & \text{si } 1 \leq t \leq n, \\ 0 & \text{si } t > n, \end{cases} \quad (4)$$

$$V(t) = \begin{cases} 1 & \text{si } 1 \leq t \leq n, \\ 0 & \text{si } t > n. \end{cases} \quad (5)$$

Se dice que la ARNN decide el lenguaje $\mathcal{L}$ si existe $r \in \mathbb{N}$, $r \geq n$, tal que:

$$G(t) = \begin{cases} 1 & \text{si } t = r \wedge \alpha \in \mathcal{L}, \\ 0 & \text{si } t \neq r \vee \alpha \notin \mathcal{L}, \end{cases} \quad (6)$$

y

$$H(t) = \begin{cases} 1 & \text{si } t = r, \\ 0 & \text{si } t \neq r, \end{cases} \quad (7)$$

3 Simulación de familias de Circuitos Booleanos por medio de ARNNs con Pesos Reales

En esta sección se presenta la relación entre familias de circuitos booleanos y ARNNs. En este artículo sólo se presenta la demostración de la simulación de familias de circuitos booleanos (uniforme o no uniforme) por medio de ARNNs con pesos reales y con función de activación σ definida por:

$$\sigma(x) = \begin{cases} 0 & \text{si } x < 0, \\ x & \text{si } 0 \leq x \leq 1, \\ 1 & \text{si } x > 1, \end{cases} \quad (8)$$

en [11, 10] se presenta también la demostración en el sentido contrario, es decir, toda ARNN con pesos reales y función de activación σ (ec. 8) puede ser simulada por una familia de circuitos booleanos; pero para

el propósito de demostrar que las ARNNs con pesos reales son un modelo de hipercomputación es suficiente con presentar la demostración en el primer sentido.

Antes de demostrar la simulación de familias de circuitos booleanos por medio de ARNNs con pesos reales se presentan algunos lemas y definiciones que servirán para construir dicha demostración. En el lema (1) se construye la ARNN $\mathcal{N}^i$ (*input network*), $\mathcal{N}^i$ codifica en base 9 la palabra $\alpha \in \{0, 1\}^*$ a ser decidida y codifica en base 2 la longitud de la palabra α ($|\alpha|$). El lema (2) construye la ARNN $\mathcal{N}^r$ (*retrieve network*), $\mathcal{N}^r$ contiene un peso real q que es la codificación en base 9 de la familia de circuitos booleanos $\mathcal{F}$ a ser simulada, $\mathcal{N}^r$ recibe como entrada la codificación en base 2 de $|\alpha|$ y produce como salida la codificación en base 9 del circuito $C_{|\alpha|} \in \mathcal{F}$. El lema (3) construye la ARNN $\mathcal{N}^s$ (*simulation network*) que recibe como entradas la codificación en base 9 de la palabra α a decidir y la codificación en base 9 del circuito $C_{|\alpha|}$ y simula la computación del circuito $C_{|\alpha|}$ con la entrada α . Por último se presenta el teorema donde se construye la ARNN $\mathcal{N}^{\mathcal{F}}$ conectando las ARNNs $\mathcal{N}^i$, $\mathcal{N}^r$ y $\mathcal{N}^s$ para decidir lenguajes simulando familias de circuitos booleanos y cumpliendo con el protocolo de entrada-salida presentado en la definición (11).

Definición 12 (Codificación de Secuencias de Dígitos en Base b). *Sea b un número natural y $\alpha = \alpha_1\alpha_2\ldots$ una secuencia finita o infinita de números naturales menores que b , la codificación de la secuencia α en base b está definida por la ecuación:*

$$(\alpha)_b = \sum_{i=1}^{|\alpha|} \frac{\alpha_i}{b^i}, \quad (9)$$

donde $|\alpha|$ puede ser ∞ ; esta codificación asegura que el valor obtenido sea un número real en el intervalo $[0, 1]$.

Definición 13 (Codificación de Secuencias Binarias). *Sea $\alpha = \alpha_1\alpha_2\ldots\alpha_n$ una secuencia binaria finita ($\alpha \in \{0, 1\}^*$), la codificación de α en el alfabeto $\Sigma = \{2, 4\}$ está dada por:*

$$e[\alpha] = \alpha'_1\alpha'_2\ldots\alpha'_n = \begin{cases} \alpha'_i = 2 & \text{si } \alpha_i = 0, \\ \alpha'_i = 4 & \text{si } \alpha_i = 1. \end{cases} \quad (10)$$

Lema 1. *Sea α una palabra sobre el alfabeto $\Sigma = \{0, 1\}$, existe una ARNN $\mathcal{N}^i$ que recibe como entradas:*

$$u_1 = \alpha 0^\infty \text{ y } u_2 = 1^{|\alpha|} 0^\infty \quad (11)$$

y produce como salidas:

$$y_1 = 0^p(e[\alpha])_9 0^\infty \text{ y } y_2 = 0^p(u_2)_2 0^\infty, \quad (12)$$

donde p es el tiempo de retardo de $\mathcal{N}^i$.

Demostración. La ARNN $\mathcal{N}^i$ presentada en la figura³ (6) con función de activación σ (ec. 8) y comenzando con valores de activación $x_1(0) = x_2(0) = \frac{1}{9}$, $x_3(0) = \frac{1}{2}$ y $x_i(0) = 0$ para $i = 4, 5, \ldots, 11$; al recibir como entradas $u_1 = \alpha 0^\infty$ y $u_2 = 1^{|\alpha|} 0^\infty$ produce como salidas $y_1 = 0^p(e[\alpha])_9 0^\infty$ y $y_2 = 0^p(u_2)_2 0^\infty$. Las neuronas x_1 y x_2 calculan la función $\frac{1}{9^{t+1}}$, donde t es el tiempo que va de computación; las neuronas x_4 y x_5 calculan las funciones $2(\frac{1}{9^t})$ y $4(\frac{1}{9^t})$ respectivamente; la neurona x_7 funciona como un acumulador, donde en cada instante de tiempo t se suma el valor de x_4 si $\alpha_{t-1} = 0$ ó se suma el valor de x_5 si $\alpha_{t-1} = 1$, x_7 calcula la función $(e[\alpha])_9$ con un instante de tiempo de retardo. La neurona x_3 calcula la función $\frac{1}{2^{t+1}}$; la neurona

³Las conexiones que no tienen especificado el peso tienen peso 1

x_6 calcula la función $(u_2)_2$. Las neuronas x_8, x_9, x_{10} y x_{11} sirven para controlar que $\mathcal{N}^i$ sólo produzca el resultado cuando la palabra α haya terminado, además sincronizan la salida de y_1 y y_2 . $\mathcal{N}^i$ computa con tiempo de retardo $p = |\alpha| + 2$. $\mathcal{N}^i$ cumple con el protocolo de entrada especificado en la definición (11), los valores de salida y_1 y y_2 serán entradas a las ARNNs $\mathcal{N}^r$ y $\mathcal{N}^s$.

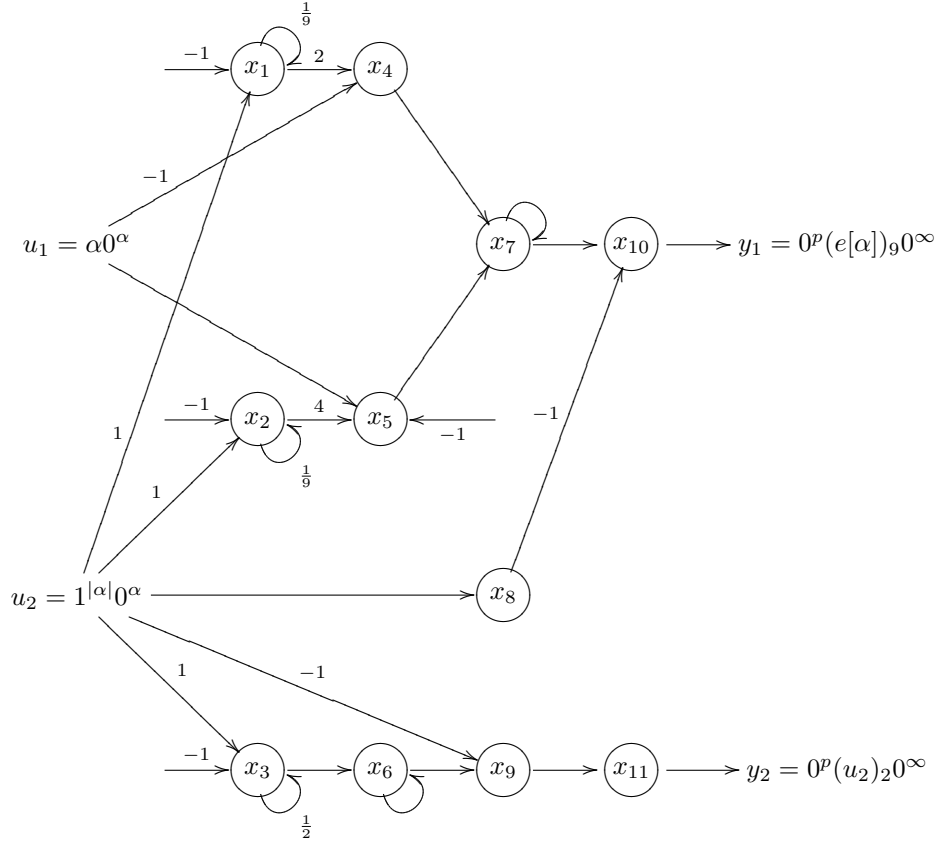


Figura 6: ARNN $\mathcal{N}^i$.

□

Definición 14 (Codificación de Circuitos Booleanos). *La codificación del circuito booleano C , denotada por $e[C]$, es una secuencia finita sobre el alfabeto $\Sigma = \{0, 2, 4, 6\}$ construida siguiendo las reglas⁴:*

1. *El circuito se codifica por niveles, comenzando por el nivel que recibe las entradas del circuito y ascendiendo en los niveles hasta llegar a las salidas del circuito. El comienzo de cada nivel se codifica con la secuencia 66.*

⁴Se adaptó la codificación dada en [11, 10] para representar entradas a las compuertas que no provienen del nivel inmediatamente anterior.

2. Dentro de cada nivel se codifican las compuertas que lo conforman en forma consecutiva, indicando el tipo de compuerta y las entradas que recibe de los niveles anteriores. La codificación de cada compuerta está compuesta por:

- (a) El símbolo 0 para indicar el comienzo de la codificación de la compuerta.
- (b) Una secuencia de dos dígitos $\{42, 44, 22\}$ para determinar el tipo de compuerta $\{\wedge, \vee, \neg\}$ respectivamente.
- (c) Las entradas a la compuerta se codifican por niveles, comenzando por el nivel anterior al nivel donde está la compuerta y descendiendo en los niveles hasta llegar al nivel donde están las entradas del circuito. El comienzo de las entradas de cada nivel se codifica con el símbolo 6, luego va una secuencia de símbolos $\{2, 4\}^w$, donde w es el número de compuertas en el nivel del que se están tomando las entradas y la posición j de esta secuencia es 4 si la compuerta que está siendo codificada recibe entradas de la compuerta j del nivel del que se están tomando las entradas, o es 2 si no.

Ejemplo 6. Sea C el circuito presentado en el ejemplo (1), la codificación de C es:

$$e[C] = 6604264422044622446604264462222660226462262222.$$

Definición 15 (Codificación de Familias de Circuitos Booleanos). La codificación de una familia de circuitos booleanos $\mathcal{F}$, denotada por $e[\mathcal{F}]$, es una secuencia infinita sobre el alfabeto $\Sigma = \{0, 2, 4, 6, 8\}$, definida por:

$$e[\mathcal{F}] = 8\bar{e}[C_1]8\bar{e}[C_2] \dots, \quad (13)$$

donde $\bar{e}[C_i]$ es la codificación $e[C_i]$ del circuito $C_i \in \mathcal{F}$ en forma invertida. Se invierte la codificación de los C_i para facilitar la construcción de la ARNN $\mathcal{N}^r$.

Lema 2. Para toda familia de circuitos booleanos $\mathcal{F} = \{C_n \mid n \in \mathbb{N}\}$ (uniforme o no uniforme) existe una ARNN $\mathcal{N}^r$ con pesos reales, tal que $\mathcal{N}^r$ recibe como única entrada:

$$u_1 = (1^n)_2 0^\infty, \quad (14)$$

y produce como salida:

$$y_1 = 0^p (e[C_n])_9 0^\infty, \quad (15)$$

donde p es el tiempo de retardo de la red $\mathcal{N}^r$.

Demostración. Sea $\mathcal{F} = \{C_n \mid n \in \mathbb{N}\}$ una familia de circuitos booleanos:

1. Se denotará por $\hat{\mathcal{F}}$ a $(e[\mathcal{F}])_9$, donde $e[\mathcal{F}]$ es una palabra sobre el alfabeto $\Sigma = \{0, 2, 4, 6\}$. $\hat{\mathcal{F}}$ es un número real en el intervalo $[0, 1]$ y:

$$\hat{\mathcal{F}} = \sum_{i=1}^{\infty} \frac{e[\mathcal{F}]_i}{9^i} = \frac{e[\mathcal{F}]_1}{9} + \sum_{i=2}^{\infty} \frac{e[\mathcal{F}]_i}{9^i}, \quad (16)$$

cuando $e[\mathcal{F}]_i = 0$ para todo $i \geq 2$ entonces $\sum_{i=2}^{\infty} \frac{e[\mathcal{F}]_i}{9^i} = 0$ y cuando $e[\mathcal{F}]_i = 8$ para todo $i \geq 2$ entonces $\sum_{i=2}^{\infty} \frac{e[\mathcal{F}]_i}{9^i} = \frac{1}{9}$, por lo tanto:

$$\widehat{\mathcal{F}} \in \left[\frac{e[\mathcal{F}]_1}{9}, \frac{e[\mathcal{F}]_1}{9} + \frac{1}{9} \right]. \quad (17)$$

2. Se define la función $\Lambda: \mathbb{R} \rightarrow [0, 1]$ por:

$$\Lambda[x] = \begin{cases} 0 & \text{si } x < 0, \\ 9x - \lfloor 9x \rfloor & \text{si } 0 \leq x \leq 1, \\ 1 & \text{si } x > 1, \end{cases} \quad (18)$$

donde $\lfloor x \rfloor$ denota parte entera de x .

Como $\widehat{\mathcal{F}}$ es un número real en el intervalo $[0, 1]$, al aplicar la función $\Lambda[x]$ a $\widehat{\mathcal{F}}$ se obtiene:

$$\Lambda[\widehat{\mathcal{F}}] = 9\widehat{\mathcal{F}} - \lfloor 9\widehat{\mathcal{F}} \rfloor = 9 \sum_{i=1}^{\infty} \frac{e[\mathcal{F}]_i}{9^i} - \left\lfloor 9 \sum_{i=1}^{\infty} \frac{e[\mathcal{F}]_i}{9^i} \right\rfloor, \quad (19)$$

como:

$$9 \sum_{i=1}^{\infty} \frac{e[\mathcal{F}]_i}{9^i} = \sum_{i=1}^{\infty} \frac{e[\mathcal{F}]_i}{9^{i-1}} = e[\mathcal{F}]_1 + \sum_{i=2}^{\infty} \frac{e[\mathcal{F}]_i}{9^{i-1}}, \quad (20)$$

y $\sum_{i=2}^{\infty} \frac{e[\mathcal{F}]_i}{9^{i-1}} < 1$, entonces:

$$\Lambda[\widehat{\mathcal{F}}] = e[\mathcal{F}]_1 + \sum_{i=2}^{\infty} \frac{e[\mathcal{F}]_i}{9^{i-1}} - e[\mathcal{F}]_1 = \sum_{i=1}^{\infty} \frac{e[\mathcal{F}]_{i+1}}{9^i}. \quad (21)$$

Por lo tanto, el resultado de aplicar la función $\Lambda[x]$ a $\widehat{\mathcal{F}}$ es eliminar el símbolo $e[\mathcal{F}]_1$ de $\widehat{\mathcal{F}}$.

3. Se define la función $\Psi: \mathbb{R} \rightarrow [0, 1]$ por:

$$\Psi[x] = \begin{cases} 0 & \text{si } x < 0, \\ 2 \lfloor \frac{9x}{2} \rfloor & \text{si } 0 \leq x \leq 1, \\ 1 & \text{si } x > 1, \end{cases} \quad (22)$$

Como $\widehat{\mathcal{F}}$ es un número real en el intervalo $[0, 1]$, al aplicar la función $\Psi[x]$ a $\widehat{\mathcal{F}}$ se obtiene:

$$\Psi[\widehat{\mathcal{F}}] = 2 \left\lfloor \frac{9\widehat{\mathcal{F}}}{2} \right\rfloor = 2 \left\lfloor \frac{9}{2} \sum_{i=1}^{\infty} \frac{e[\mathcal{F}]_i}{9^i} \right\rfloor, \quad (23)$$

como:

$$\frac{9}{2} \sum_{i=1}^{\infty} \frac{e[\mathcal{F}]_i}{9^i} = \sum_{i=1}^{\infty} \frac{e[\mathcal{F}]_i}{2(9^{i-1})} = \frac{e[\mathcal{F}]_1}{2} + \sum_{i=2}^{\infty} \frac{e[\mathcal{F}]_i}{2(9^{i-1})} \quad (24)$$

y $\sum_{i=2}^{\infty} \frac{e[\mathcal{F}]_i}{2(9^{i-1})} < 1$, entonces:

$$\Psi[\widehat{\mathcal{F}}] = 2 \frac{e[\mathcal{F}]_1}{2} = e[\mathcal{F}]_1. \quad (25)$$

Por lo tanto, la función $\Psi[x]$ sirve para obtener el símbolo $e[\mathcal{F}]_1$ de $\widehat{\mathcal{F}}$.

4. Se define la función $\widetilde{\Psi}: \mathbb{R} \rightarrow [0, 1]$ por:

$$\widetilde{\Psi}[x] = 2(\sigma(9x-1) + \sigma(9x-3) + \sigma(9x-5) + \sigma(9x-7)) = 2 \sum_{j=0}^3 \sigma(9x - (2j+1)), \quad (26)$$

Al aplicar la función $\widetilde{\Psi}[x]$ a $\widehat{\mathcal{F}}$ para los diferentes valores de $e[\mathcal{F}]_1 \in \{0, 2, 4, 6, 8\}$ y con base en la ecuación (17) se obtienen los valores de la tabla (1).

$e[\mathcal{F}]_1$	$\widehat{\mathcal{F}}$	$\sigma(9\widehat{\mathcal{F}}-1)$	$\sigma(9\widehat{\mathcal{F}}-3)$	$\sigma(9\widehat{\mathcal{F}}-5)$	$\sigma(9\widehat{\mathcal{F}}-7)$	$\sum_{j=0}^3 \sigma(9\widehat{\mathcal{F}} - (2j+1))$	$\widetilde{\Psi}[\widehat{\mathcal{F}}]$
0	$[0, \frac{1}{9}]$	0	0	0	0	0	0
2	$[\frac{2}{9}, \frac{3}{9}]$	1	0	0	0	1	2
4	$[\frac{4}{9}, \frac{5}{9}]$	1	1	0	0	2	4
6	$[\frac{6}{9}, \frac{7}{9}]$	1	1	1	0	3	6
8	$[\frac{8}{9}, 1]$	1	1	1	1	4	8

Cuadro 1: Valores de $\widetilde{\Psi}[x]$ para $\widehat{\mathcal{F}}$.

En la tabla (1) se puede observar que $\widetilde{\Psi}[\widehat{\mathcal{F}}] = e[\mathcal{F}]_1$ para todo $e[\mathcal{F}]_1$. Por lo tanto, La función $\Psi[\mathcal{F}]$ puede ser aproximada mediante la función $\widetilde{\Psi}[\mathcal{F}]$.

5. Se define la función $\widetilde{\Lambda}: \mathbb{R} \rightarrow [0, 1]$ por:

$$\begin{aligned} \widetilde{\Lambda}[x] &= (\sigma(9x) + \sigma(9x-2) + \sigma(9x-4) + \sigma(9x-6) + \sigma(9x-8)) - \\ &\quad (\sigma(9x-1) + \sigma(9x-3) + \sigma(9x-5) + \sigma(9x-7)) \\ &= \sum_{j=0}^4 \sigma(9\widehat{\mathcal{F}} - 2j) - \sum_{j=0}^3 \sigma(9\widehat{\mathcal{F}} - (2j+1)) \\ &= \sum_{j=0}^8 ((-1)^j \sigma(9x - j)), \end{aligned} \quad (27)$$

Al aplicar la función $\tilde{\Lambda}[x]$ a $\hat{\mathcal{F}}$ para los diferentes valores de $e[\mathcal{F}]_1 \in \{0, 2, 4, 6, 8\}$, con base en la ecuación (17) y que $(\sigma(9x-1) + \sigma(9x-3) + \sigma(9x-5) + \sigma(9x-7)) = \sum_{j=0}^3 \sigma(9\hat{\mathcal{F}} - (2j+1)) = \frac{e[\mathcal{F}]_1}{2}$ (tabla 1) se obtienen los valores de la tabla (2).

$e[\mathcal{F}]_1$	$\hat{\mathcal{F}}$	$\sigma(9\hat{\mathcal{F}})$	$\sigma(9\hat{\mathcal{F}}-2)$	$\sigma(9\hat{\mathcal{F}}-4)$	$\sigma(9\hat{\mathcal{F}}-6)$	$\sigma(9\hat{\mathcal{F}}-8)$	$\sum_{j=0}^4 \sigma(9\hat{\mathcal{F}}-2j)$	$\tilde{\Lambda}[\hat{\mathcal{F}}]$
0	$[0, \frac{1}{9}]$	$9\hat{\mathcal{F}}$	0	0	0	0	$9\hat{\mathcal{F}}$	$9\hat{\mathcal{F}}$
2	$[\frac{2}{9}, \frac{3}{9}]$	1	$9\hat{\mathcal{F}}-2$	0	0	0	$9\hat{\mathcal{F}}-1$	$9\hat{\mathcal{F}}-2$
4	$[\frac{4}{9}, \frac{5}{9}]$	1	1	$9\hat{\mathcal{F}}-4$	0	0	$9\hat{\mathcal{F}}-2$	$9\hat{\mathcal{F}}-4$
6	$[\frac{6}{9}, \frac{7}{9}]$	1	1	1	$9\hat{\mathcal{F}}-6$	0	$9\hat{\mathcal{F}}-3$	$9\hat{\mathcal{F}}-6$
6	$[\frac{6}{9}, \frac{7}{9}]$	1	1	1	1	$9\hat{\mathcal{F}}-8$	$9\hat{\mathcal{F}}-4$	$9\hat{\mathcal{F}}-8$

Cuadro 2: Valores de $\tilde{\Lambda}[x]$ para $\hat{\mathcal{F}}$.

En la tabla (2) se puede observar que $\tilde{\Lambda}[\hat{\mathcal{F}}] = 9\hat{\mathcal{F}} - \lfloor 9\hat{\mathcal{F}} \rfloor = \Lambda[\hat{\mathcal{F}}] = \sum_{i=1}^{\infty} \frac{e[\mathcal{F}]_{i+1}}{9^i}$ para todo $e[\mathcal{F}]_1$. Por lo tanto, La función $\Lambda[\hat{\mathcal{F}}]$ puede ser aproximada mediante la función $\tilde{\Lambda}[\hat{\mathcal{F}}]$.

6. Para extraer $(e[C_n])_9$ de $\hat{\mathcal{F}}$ se define el algoritmo:

Procedimiento Recuperar($\hat{\mathcal{F}}, n$)

Comienzo

 contador $\leftarrow 0$,

$y \leftarrow 0$,

$z \leftarrow \hat{\mathcal{F}}$,

 Mientras (contador $< n$) Haga

 Si ($\Psi[z] = 8$) Entonces

 contador $\leftarrow$ contador + 1,

 Fin Si,

$z \leftarrow \Lambda[z]$,

 Fin Mientras,

 Mientras ($\Psi[z] < 8$) Haga

$z \leftarrow \Lambda[z]$,

$y \leftarrow \frac{1}{9}(y + \Psi[z])$, // acumula $\bar{e}[C_n]$ invirtiéndola de nuevo y obteniendo $(e[C_n])_9$.

 Fin Mientras,

 Retorne(y),

Fin Recuperar.

7. La ARNN $\mathcal{N}^r$ presentada en la figura⁵ (7) con función de activación σ (ec. 8) y comenzando con valor de activación 0 para todas las neuronas, se basa en el algoritmo de recuperación presentado en el paso anterior y en la definición de las funciones $\tilde{\Psi}[x]$ (ec. 26) y $\tilde{\Lambda}[x]$ (ec. 27) para satisfacer el lema (2). La neurona x_{10} se corresponde con la variable z del algoritmo, al principio x_{10} recibe $\hat{\mathcal{F}}$ por medio del peso proveniente de x_9 , al x_{10} enviar su valor a las neuronas $x_i, i = 0, 1, \dots, 8$; éstas calculan la función

⁵Las conexiones que no tienen especificado el peso tienen peso 1.

$\sigma(9\widehat{\mathcal{F}} - i)$, x_{10} recibe de nuevo los valores calculados por estas neuronas computando la función $\tilde{\Lambda}[\widehat{\mathcal{F}}]$ de acuerdo a la ecuación (27). Las neuronas x_{11} y x_{12} realizan la instrucción $y \leftarrow \frac{1}{9}(y + \Psi[z])$, x_{11} calcula el valor de $\frac{\Psi[z]}{9}$ de acuerdo a la ecuación (26) y le suma a esto $\frac{y}{9}$ por medio del enlace proveniente de la neurona x_{12} . Las neuronas x_{13} , x_{14} y x_{15} simulan el contador, disminuyendo n en base 2 cada vez que se encuentra un 8 en la codificación ($x_7 = 1$) hasta llegar a $n = 0$, limpiando el acumulador para la variable y (neurona x_{11}) mientras $n > 0$. x_{16} sólo toma el valor de y de la neurona x_{12} cuando $n = 0$ ($x_{13} = 0$) y se encuentra un 8 en la codificación ($x_7 = 1$). De este modo $\mathcal{N}^r$ recibe como entrada $u_1 = (1^n)_2 0^\infty$ y produce como salida $y_1 = 0^p(e[C_n])_9 0^\infty$.

□

Definición 16 (Máquinas de Turing k -cintas). *Una máquina de Turing k -cintas es una máquina de Turing que opera simultáneamente sobre k cintas de trabajo, existiendo una cabeza de lectura-escritura para cada cinta.*

Formalmente una máquina de Turing k -cintas, para $k > 1$, está definida por la estructura matemática:

$$\mathcal{MT} = \langle Q, \Sigma, M, I \rangle, \quad \text{donde :}$$

Q, Σ y M son los mismos que para una máquina de Turing (definición 6).

$I = \{i_0, i_1, \dots, i_p\}$: Conjunto finito de intrucciones de la forma $i_j = q_m s_{m_i} \dots s_{m_k} s_{n_1} \dots s_{n_k} m_1 \dots m_k q_n$, donde $q_m \in Q$, $q_n \in Q \cup \{q_Y, q_N\}$, $s_{m_i}, s_{n_i} \in \Sigma$ y $m_i \in M$. Las instrucciones también pueden ser vistas como una relación de $Q \times \Sigma^k$ en $\Sigma^k \times M^k \times Q \cup \{q_Y, q_N\}$, si existe más de una instrucción para alguna tupla $q_m s_{m_i} \dots s_{m_k} \in \{Q \times \Sigma^k\}$ la máquina de Turing k -cintas es ‘no determinista’, si en las intrucciones no se repite la tupla $q_m s_{m_i} \dots s_{m_k}$ la máquina de Turing k -cintas es ‘determinista’ [8].

Lema 3. Sea α una palabra sobre el alfabeto $\Sigma = \{0, 1\}$ y C un circuito booleano, existe una ARNN $\mathcal{N}^s$ que recibe como entradas:

$$u_1 = (e[C])_9 0^\infty \text{ y } u_2 = (e[\alpha])_9 0^\infty \quad (28)$$

y produce como salidas:

$$y_1 = 0^p \phi_c(\alpha) 0^\infty \text{ y } y_2 = 0^p 10^\infty, \quad (29)$$

donde $\phi_c(\alpha) = 1$ si el circuito C produce 1 con la entrada α ó $\phi_c(\alpha) = 0$ si el circuito C produce 0 con la entrada α y p es el tiempo de retardo de la red $\mathcal{N}^s$.

Demostración. Todo circuito booleano puede ser simulado por una máquina de Turing con tres cintas, en la primera cinta se almacena la codificación del circuito, en la segunda cinta se almacena la codificación de la entrada y la salida y la tercera cinta sirve para llevar el control del nivel del circuito booleano que se está calculando. Además, cualquier máquina de Turing puede ser simulada por una ARNN con pesos racionales [7, 12, 10]. Por lo tanto $\mathcal{N}^s$ puede ser construida. □

Teorema 2. *Todo lenguaje decidido por una familia de circuitos booleanos (uniforme o no uniforme) puede ser decidido por una ARNN con pesos reales [11, 10].*

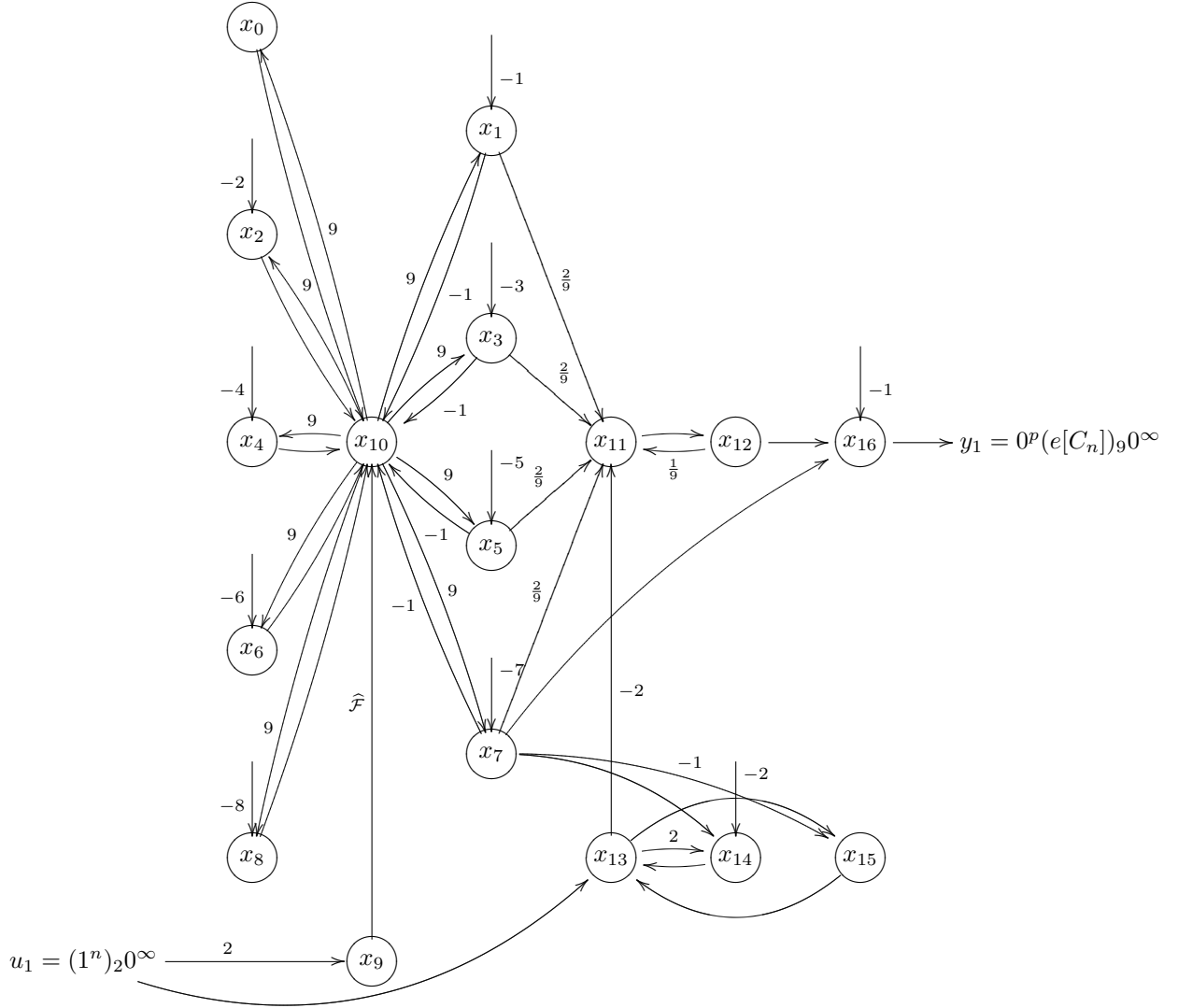


Figura 7: ARNN $\mathcal{N}^r$.

Demostración. Sea $\alpha = \alpha_1\alpha_2\ldots\alpha_n$ una palabra sobre el alfabeto $\Sigma = \{0, 1\}$, $\mathcal{F} = \{C_n \mid n \in \mathbb{N}\}$ una familia de circuitos booleanos y $\mathcal{L} \subseteq \{0, 1\}^*$ un lenguaje, tal que $\mathcal{F}$ decide $\mathcal{L}$. La ARNN $\mathcal{N}^{\mathcal{F}}$ (*circuit familie network*) presentada en la figura⁶ (8), donde $\mathcal{N}^i$, $\mathcal{N}^r$ y $\mathcal{N}^s$ son las ARNNs especificadas en los lemas (1), (2) y (3) respectivamente, al recibir como entradas $u_1 = \alpha 0^\infty$ y $u_2 = 1^{|\alpha|} 0^\infty$ produce como salidas $y_1 = 0^p \phi_c(\alpha) 0^\infty$ y $y_2 = 0^p 10^\infty$, donde $\phi_c(\alpha) = 1$ si $\alpha \in \mathcal{L}$ ó $\phi_c(\alpha) = 0$ si $\alpha \notin \mathcal{L}$. Las neuronas $x_{28}, x_{29}, \dots, x_{33}$ sirven para sincronizar las entradas a la ARNN $\mathcal{N}^s$. $\mathcal{N}^{\mathcal{F}}$ cumple con el protocolo de entrada-salida presentado en la definición (11).

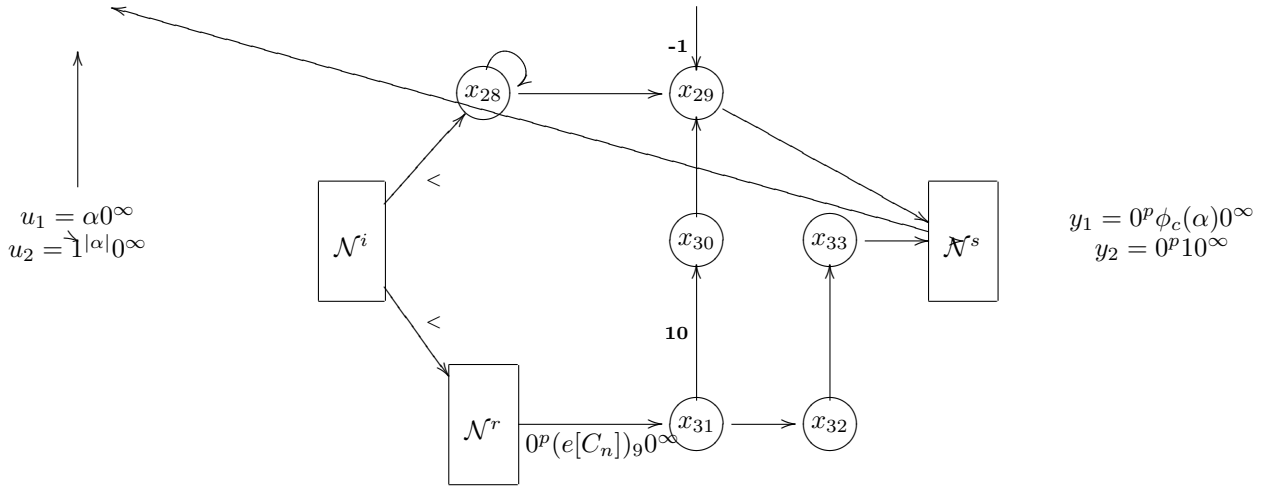


Figura 8: ARNN $\mathcal{N}^{\mathcal{F}}$.

□

4 Conclusiones

Cualquier familia de circuitos booleanos uniforme $\mathcal{F} = \{C_n \mid n \in \mathbb{N}\}$ puede ser simulada por una máquina de Turing, debido a que se puede construir la máquina de Turing $\mathcal{M}$ que recibiendo como entrada 1^n produzca como salida C_n en alguna codificación (definición 4) y cualquier circuito booleano puede ser simulado por una máquina de Turing de tres cintas (lema (3)). Por lo tanto todos los lenguajes decididos por familias de circuitos booleanos uniformes son lenguajes recursivos.

La computabilidad particiona los números reales en números reales computables y números reales no computables [1, 9, 16]. Para que una familia de circuitos booleanos decida un lenguaje no recursivo y se convierta en un modelo de hipercomputación, es necesario que la familia de circuitos booleanos sea no uniforme, esto implica que la secuencia infinita de codificación de la familia de circuitos booleanos (definición 15) no pueda ser generada por medio de una máquina de Turing, por lo tanto el número real $\hat{\mathcal{F}}$ que codifica

⁶Los valores que aparecen en negrillas (**-1**, **10**) son los pesos de las conexiones, las demás conexiones tiene peso **1**.

la familia de circuitos booleanos en base 9 debe ser un número real no computable.

En la ARNN $\mathcal{N}^{\mathcal{F}}$ el peso real $\widehat{\mathcal{F}}$ de la subred $\mathcal{N}^r$ es el que contiene la codificación en base 9 de la familia de circuitos booleanos a simular, por lo tanto, para que $\mathcal{N}^{\mathcal{F}}$ simule familias de circuitos booleanos no uniformes y pueda ser un modelo de hipercomputación es necesario que el peso $\widehat{\mathcal{F}}$ sea no computable.

Por otro lado, cuando los pesos en una ARNN con función de activación σ (ec. 8) pueden ser cualquier número real, las neuronas pueden tomar un número infinito no contable de valores de activación ($0 \leq x \leq 1 \in \mathbb{R}$), esto hace que la ARNN pueda estar en un espacio continuo de ‘configuraciones’, entendiendo por configuración una combinación de los valores de activación de las neuronas.

Agradecimientos

Este artículo fue financiado por la universidad EAFIT, bajo el proyecto de investigación número 817424.

Referencias

- [1] DOUGLAS S. BRIDGES. “Computability: A Mathematical Sketchbook”, tomo 146 de “Graduate Texts in Mathematics”. New York: Springer-Verlag (1994).
- [2] B. JACK COPELAND. The broad conception of computation. *American Behavioral Scientist* **40**, 690–716 (1997). Preprint: www.phil.canterbury.ac.nz/philsite/people/jack_copeland [15-May-1999].
- [3] B. JACK COPELAND Y RICHARD SYLVAN. Beyond the universal Turing machine. *Australasian Journal of Philosophy* **77**, 44–66 (1999).
- [4] MARTIN DAVIS. “Computability and unsolvability”. New York: Dover Publications, Inc. (1982).
- [5] RAÚL GÓMEZ MARÍN Y ANDRÉS SICARD RAMÍREZ. “Informática teórica: Elementos propedeúticos”. Medellín: Fondo Editorial U. EAFIT (2001).
- [6] MARVIN L. MINSKY. “Computation: finite and infinite machines”. Englewood Cliffs, N.J.: Prentice-Hall, Inc. (1967).
- [7] J. P. NETO, H. T. SIEGELMANN, J. F. COSTA Y C. P. ARAUJO. Turing universality of neural nets (revisited). Publicado en: Lecture notes in Computer Science: computer aided system theory. Edited by F. Pichler and R. Moreno Diaz. Eprint: www.cs.math.ist.utl.pt/cgi-bin/uncgi/bib2html.tcl?author=fgc (1977).
- [8] CHRISTOS PAPADIMITRIOU. “Computational complexity”. Reading: Addison-Wesley Publishing Company (1994).
- [9] ANDRÉS SICARD Y MARIO VÉLEZ. Algunos comentarios sobre los números reales computables. Eprint: sigma.eafit.edu.co:90/~asicard/archivos/mt-reales.ps.gz (2000).
- [10] HAVA T. SIEGELMANN. “Neural networks and analog computation. Beyond the Turing limit”. Progress in Theoretical Computer Science. Boston: Birkhäuser (1999).

- [11] HAVA T. SIEGELMANN Y EDUARDO D. SONTAG. Analog computation via neural networks. *Theoretical Computer Science* **131**(2), 331–360 (1994).
- [12] HAVA T. SIEGELMANN Y EDUARDO D. SONTAG. On computational power of neural networks. *Journal of Computer and System Sciences* **50**(1), 132–150 (1995).
- [13] HOWARD STRAUBING. “Finite automata, formal logic, and circuit complexity”. Boston: Birkhäuser (1994).
- [14] ALAN M. TURING. On computable numbers, with an application to the Entscheidungsproblem. *Proc. London Math. Soc.* **42**, 230–265 (1936-7). A correction, *ibid*, vol. 43, no. 2198, p. 544–546, 1937.
- [15] INGO WEGENER. “The complexity of boolean functions”. Stuttgart: John Wiley & Sons Ltd, and B. G. Teubner, Stuttgart (1987). Eprint: www.eccc.uni-trier.de/eccc-local/ECCC-Books/wegener_book_readme.html.
- [16] KLAUS WEIHRAUCH. “Computable Analysis: An Introduction”. Berlin: Springer-Verlag (2000). Eprint: www.informatik.fernuni-hagen.de/import/thi1/klaus.weihsrauch/book.html.